

5. System Software

5.1. Operating System

- Need for OS
 - A set of programs designed to run in the background on a computer system which
 - Controls operation of computer system
 - Provides a user interface
 - Controls how computer responds to user's requests
 - Controls how hardware communicate
 - Provides an environment in which application software can be executed
 - OS hardware is unusable without an OS, as the OS acts as an interface since it controls communication between user and hardware

Key Management Tasks

- (Main) Memory Management
 - Memory protection to ensure 2 programs do not try to use same memory space
 - Paging
 - Use of virtual memory
- File Management
 - Provides file naming conventions
 - Maintains a directory structure
 - Allocates space to particular files

- Security Management
 - Provides usernames & passwords
 - Ensures data privacy
 - Prevents unauthorized access
 - Carries out automatic backup
- Hardware (input/output/peripherals) Management
 - Installation of appropriate driver software
 - Controls access to data sent to and from peripherals
 - Receives & handles interrupts from hardware devices
- Process Management
 - Enables multiprogramming and multitasking
 - Resolution of conflicts when 2 or more processes requires the same resource
 - E.g. via Round-robin method

Utility Software

- Disk Formatter
 - Prepares a hard disk to allow data to be stored on it
 - Deletes any existing data on disk
 - Performs formatting, process where computer 'draws lines' on disk surface to split it into small areas
- Virus checker
 - Checks for and then removes any viruses found
 - Constantly checks all incoming and outgoing files
- Defragmentation Software
 - Files can be big so have to be stored in multiple sectors, which can result in fragmentation (contents of file scattered across >2 non-contiguous sectors)
 - Fragmentation slows down disk access and thus the performance of the entire computer.
 - Defragmenting software works by physically reorganizing disk contents (files) such that they are stored in contiguous sectors.
 - This defragmentation reduces number of movements of the read/write heads require to access the disk contents, hence increasing computer performance
 - The defragmentation also creates larger contiguous free space regions
- Disk contents analysis/disk repair software
 - Software utility for visualization of disk space usage
 - Gets size for each folder and files, and generates a graphical chart showing disk usage distribution according to folders or other user defined criteria.
 - Allows disk to report errors (e.g. "bad sector")
 - Software will attempt to offer a solution
- File Compression
 - Reduces file size by removing redundant data in files
 - Causes improvements in the computer's performance by reducing the data that needs to be stored

- Back-up Software
 - Makes copy of files on another storage medium in the event of a hard drive failure, user error, disaster or accident.
 - Should be a regular process
 - Can provide synchronization between devices

Program Libraries

- Pre-written code that can be linked to a software under development without any amendments
- Can perform common or complex tasks
- Takes the form of classes
- Benefits:
 - Saves time: less code needs to be written
 - Smaller testing time: pre-tested and used by others
 - Library file is a complex algorithm which the user does not need to understand to use it
- Dynamic Link Library (DLL) files
 - Shared library file that contains code and data
 - Code saved separately from the main .EXE file, reducing the .EXE file's size
 - Code only loaded to main memory when required
 - DLL file can be made available to several applications simultaneously, thus reducing strain on memory
 - DLL files act as modules in more complex programs, making it easier to install and run updates

5.2. Language Translators

- Assembler
 - Software that translates assembly language statements into machine code (binary) for execution
 - The mnemonics used translates into machine opcodes
 - Process simple because assembly language has a one-to-one relationship with machine code.
- Compiler and Interpreter

Compiler	Interpreter
Translates a high-level language program to machine code.	Translates and executes a high-level language program, line-by-line.
Creates a .exe file which can be easily distributed.	No .exe file created.
Once compiled, .exe file does not need to be compiled again, resulting in faster execution.	Execution very slow – translated each time program run.
Reports all errors at the end of compilation: difficult to locate errors. ∴ development process long.	Debugging easier/faster, since it stops translating when it reaches an error. This allows real time error correction.
Only be produced when all errors are fixed.	Can run program any time, even before code finished.
Used when development is completed.	Used during development.

- One Pass Assembler
 - A one-pass assembler is a type of assembler that processes the source code in a single scan. During this scan, it defines symbols and literals, storing them in the symbol and literal tables, respectively. It keeps track of the location counter to assign memory addresses to variables and instructions, processes pseudo-operations as they appear, and simultaneously translates the source code into machine code, effectively generating the object code while resolving addresses and symbolic references on the go.
- Two Pass Assembler
 - A two-pass assembler is a program that reads the source code twice to generate object code. In the first pass, it scans the code to define symbols, assign memory addresses, and gather information about literals and variables. In the second pass, it converts symbolic op-codes into their corresponding numeric op-codes, resolves symbol values, generates data for literals, and produces the final object code by fully translating the source code.
- Two-step translation
 - Java and some other high level language programs may require two-step translation, i.e., they will be partially compiled and partially interpreted
 - Java code first translated to bytecode by Java compiler
 - Bytecode finally interpreted by the Java Virtual Machine to produce machine code

NO

- Integrated Development Environment (IDE) features
 - Coding
 - Context-sensitive prompts: Displays choice of keywords and available identifiers appropriate at current insertion point and provides choices in alphabetical order
 - Highlights undeclared/unassigned variable identifiers
 - Initial Error Detection
 - Dynamic syntax checks: Automatic checking and highlighting of syntax errors, as soon as line typed
 - Type checking & parameter checking
 - Presentation
 - Prettyprint: Automatic indentation and color-coding of keywords
 - Expand and Collapse code blocks: Saves excessive scrolling if collapsed, and easy to see global variable declarations and main program body when collapsed
 - Debugging
 - Single stepping: Executes program line-by-line to see the effect of each statement on variables
 - Breakpoints: Pauses program at a specific line to ensure program operates correctly up to that line
 - Variables/expressions Report Window: Monitors variables for comparing values.